# Sequential Constraint Injection for Constrained Multi-Agent Reinforcement Learning

Max Bourgeat, Quentin Cappart, Antoine Legrain

CORS 2026

POLYTECHNIQUE MONTRÉAL
TECHNOLOGICAL UNIVERSITY

CORS · SCRO

# The Problem

# Railway environment

**Goal** : “Operational railway decision support based on reinforcement learning”

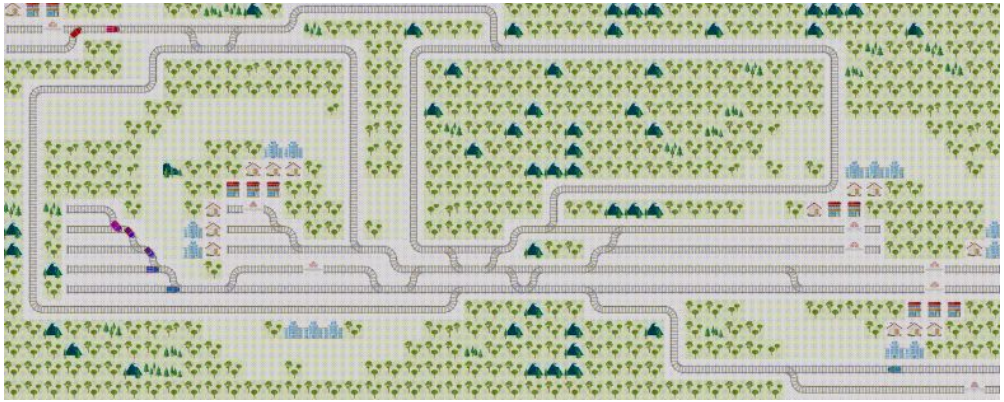
**Railway environment** : dynamic/stochastic, complex, “high-consequence environment”

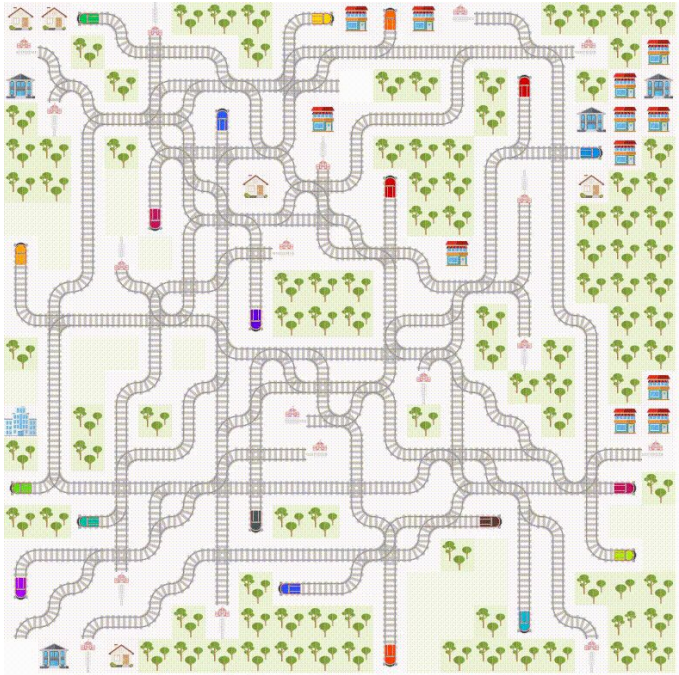
**Upstream decision-making** : train routing planning

**Real-time decision-making** : train routing re-optimization

**Reinforcement learning** : learning paradigm based on trial and error

# Railway environment

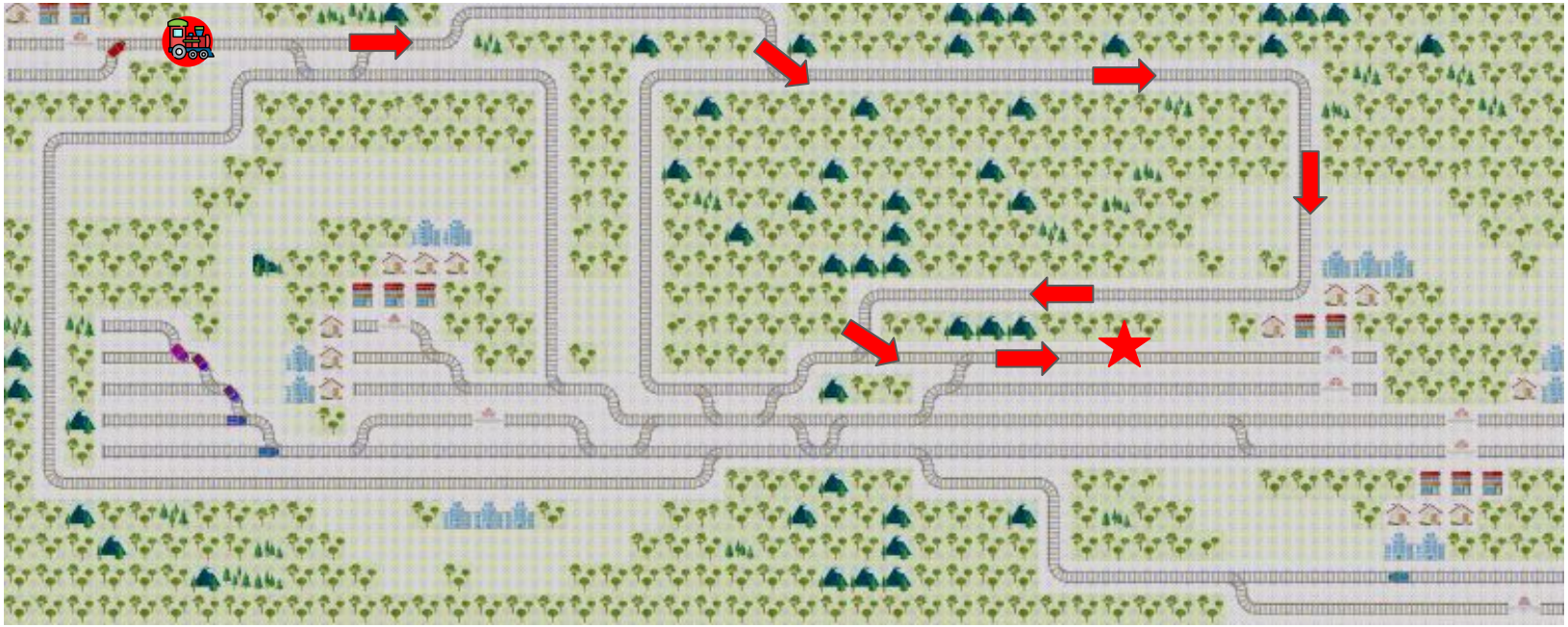




**Flatland** Mohanty et al. 2020

Simulation environment of a railway network

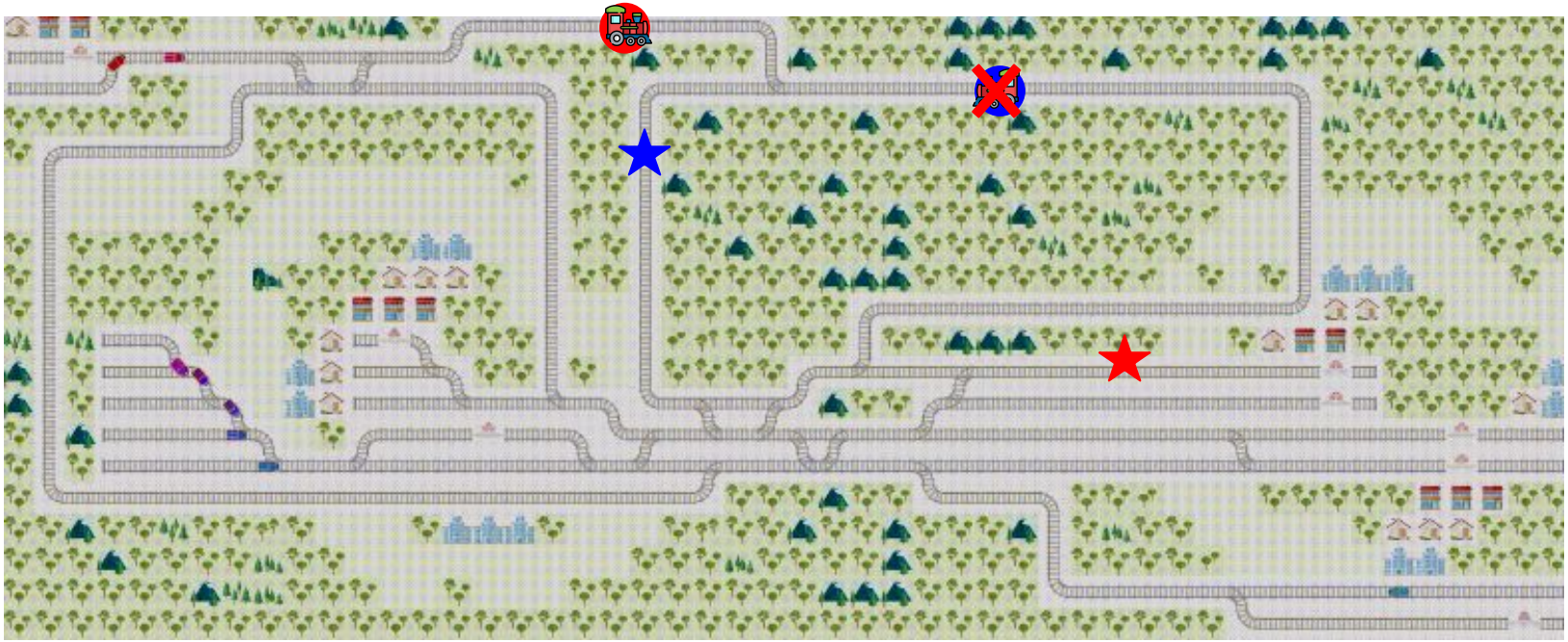
# Worst nightmare = Deadlocks



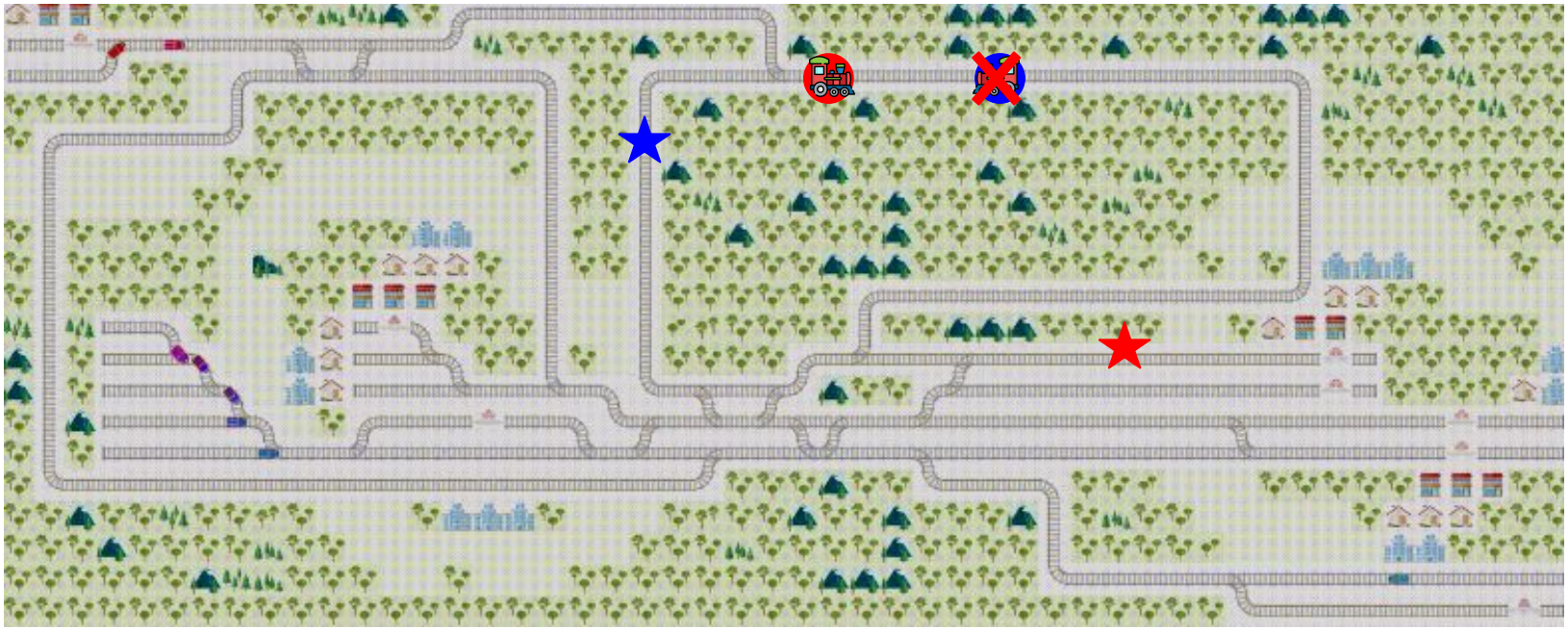
# Worst nightmare = Deadlocks

# Worst nightmare = Deadlocks

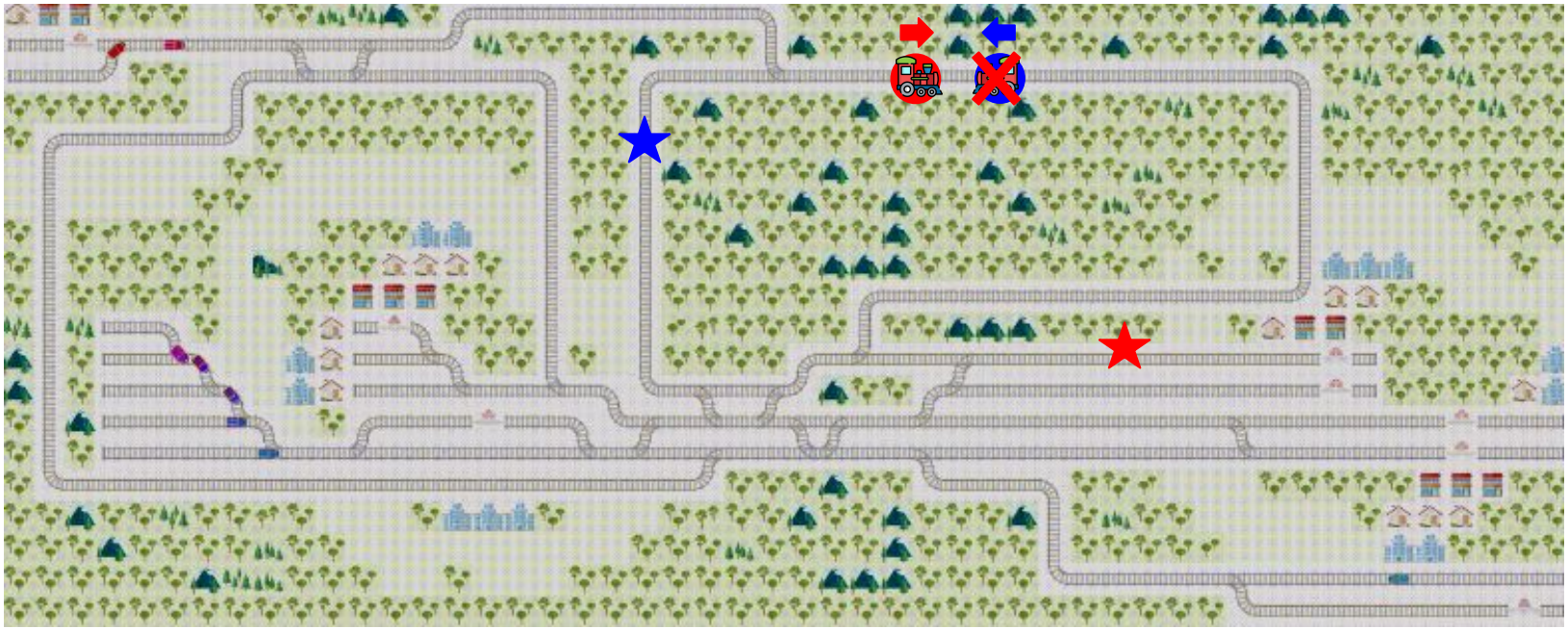
# Worst nightmare = Deadlocks



# Worst nightmare = Deadlocks



# Worst nightmare = Deadlocks



# Worst nightmare = Deadlocks

# Worst nightmare = Deadlocks

# Our mission & Contributions

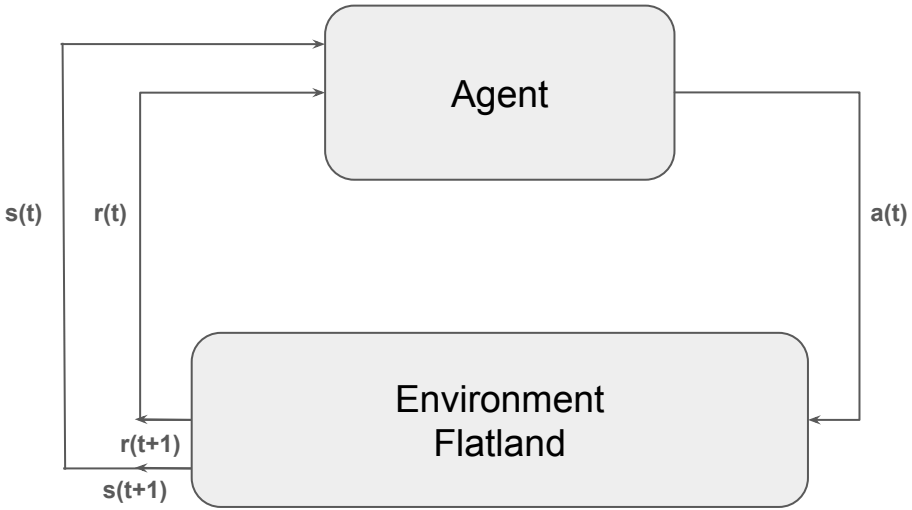
Maximum of trains at destination **AND** behaviors controlled

(deadlocked, noise disturbance, stops, …)

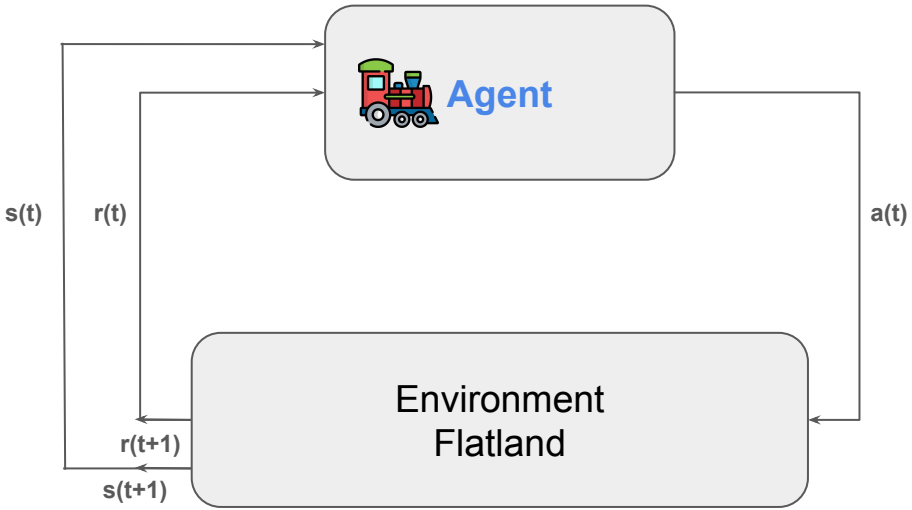
The end does **NOT** justify the mean

**Contributions:**
Primal-Dual Algorithm for Multi-Agent Constrained Reinforcement Learning
Sequential Constraint Injection Method
New AI SOTA performances on Flatland open benchmark

# Reinforcement Learning

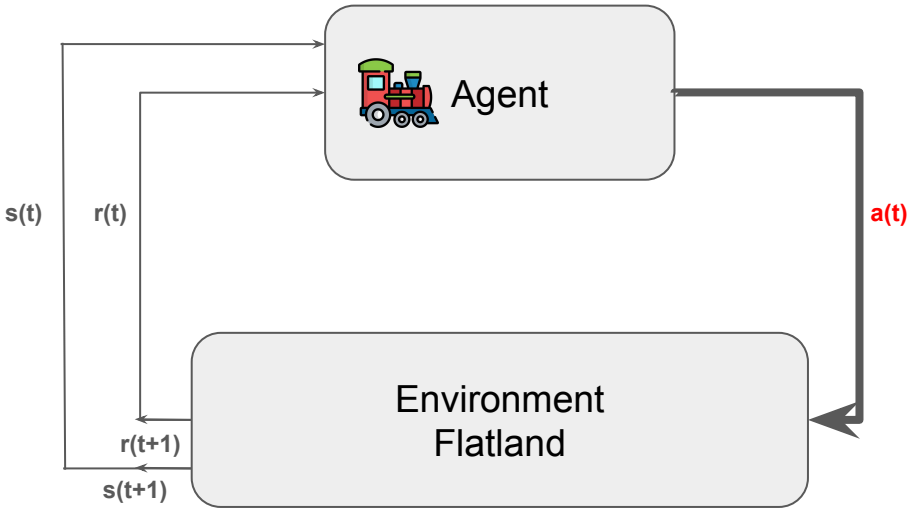


# Reinforcement Learning




**Agent : train**

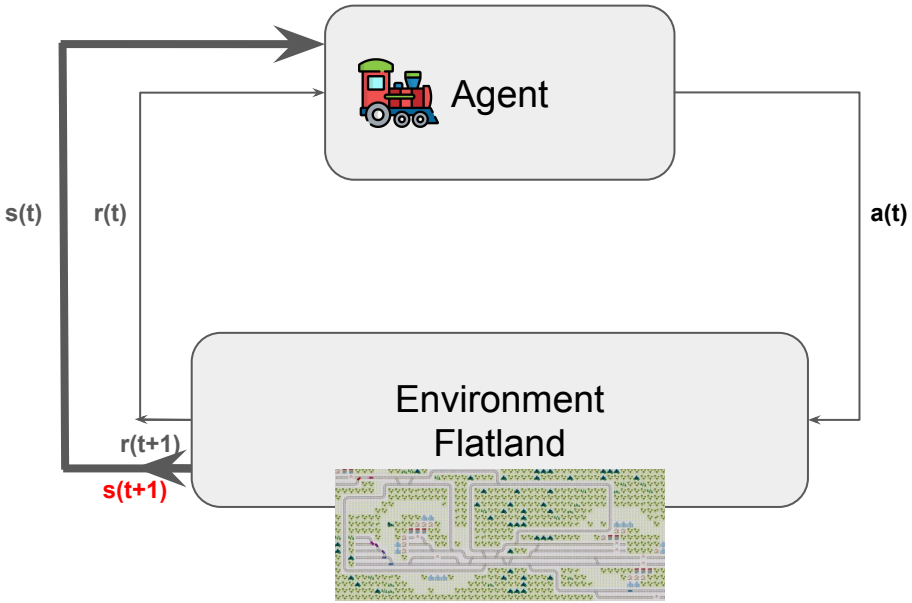
# Reinforcement Learning




Agent : train

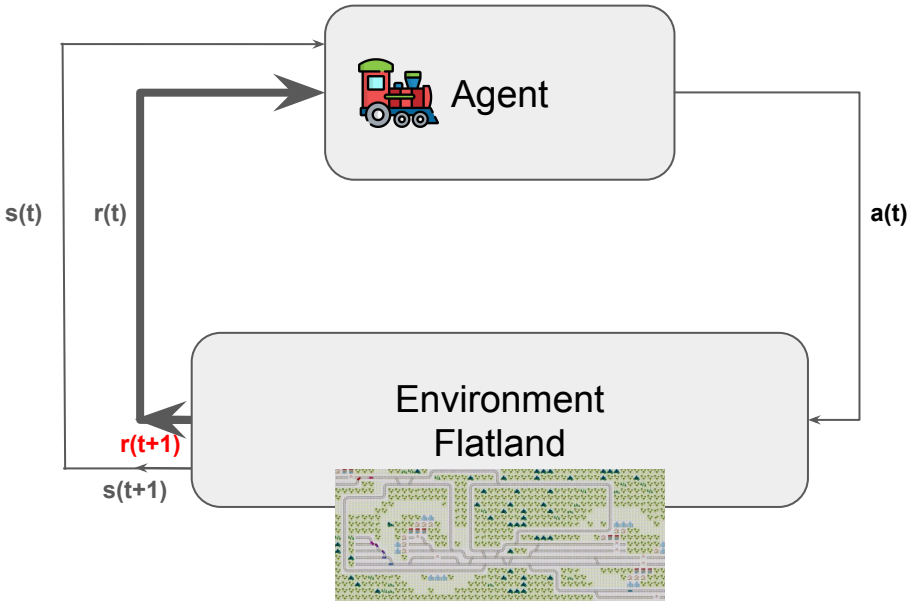
**Action : Left, Right, Forward, Stop**

# Reinforcement Learning



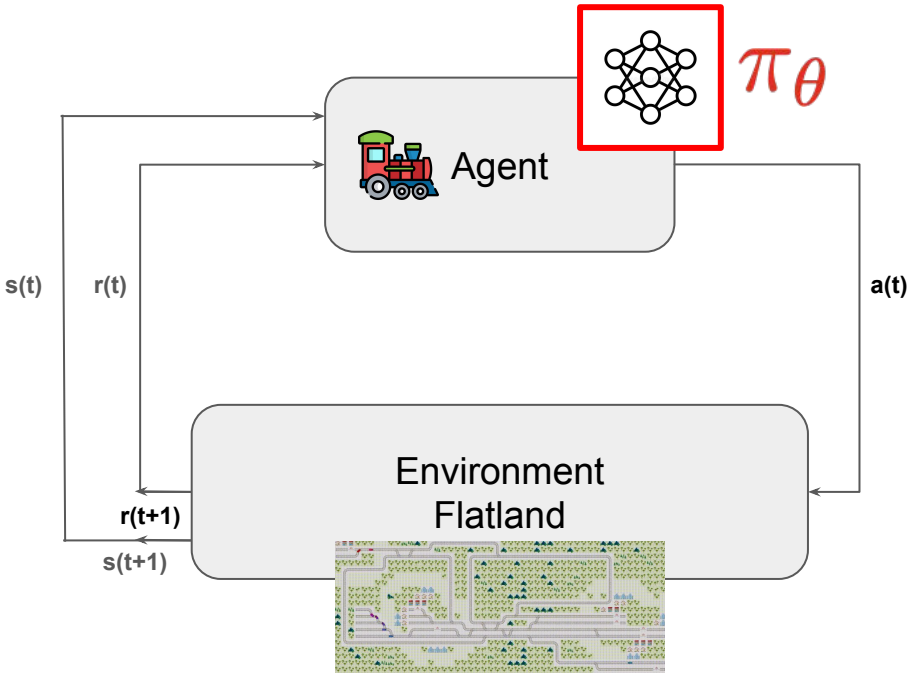

Agent : train
Action : Left, Right, Forward, Stop
**State : observation of environment**

# Reinforcement Learning



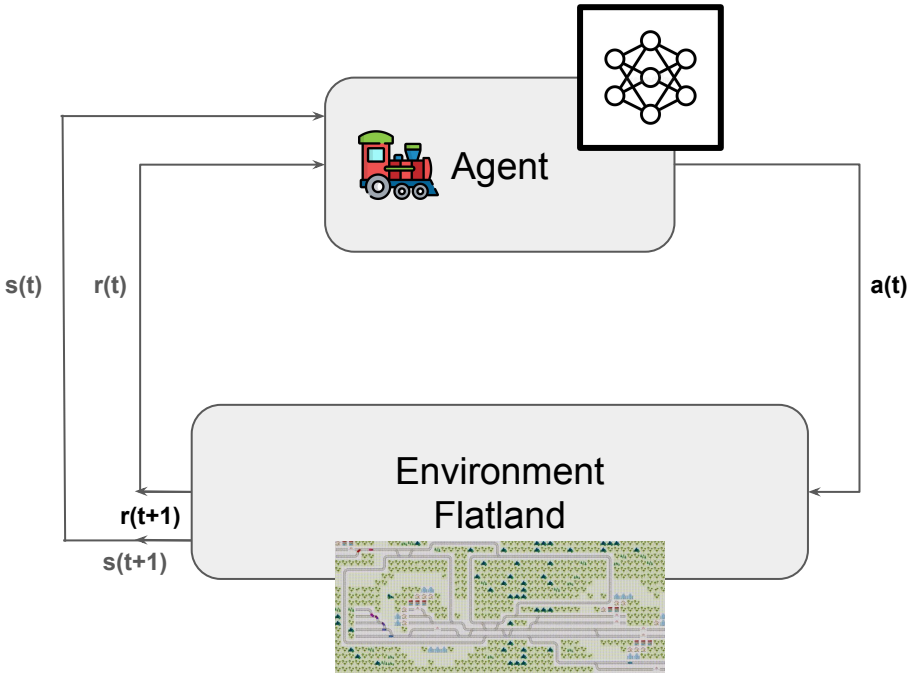

Agent : train
Action : Left, Right, Forward, Stop
State : observation of environment
**Reward : closer to destination**

# Reinforcement Learning




Agent : train
Action : Left, Right, Forward, Stop
State : observation of environment
Reward : closer to destination

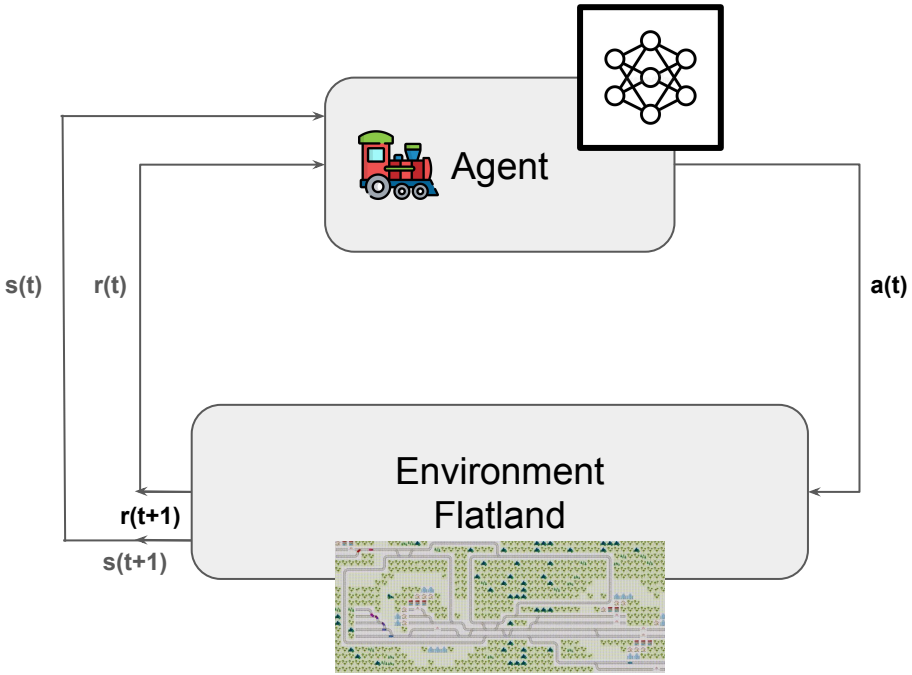
# Reinforcement Learning




**Reward is good but how to constraint behavior ?**

Egorov et al. 2022
Bourgeat et al. 2026

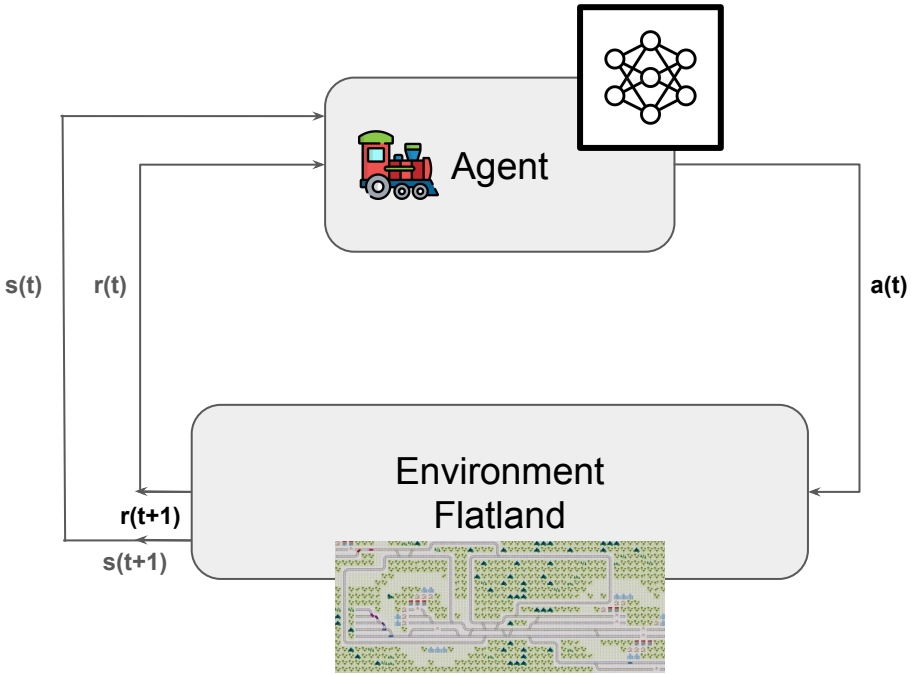
# Reinforcement Learning




**Real world** ⟶ **Constraints**

- Collision
- Noise disturbance
- Unnecessary stops
- …

# Reinforcement Learning




Real world ⟶ Constraints

**Constrained RL**

# Constrained RL, what is it ?

**RL**

$$\pi^* = \arg\max_{\pi \in \Pi} J_R(\pi)$$

find the **best policy**

# Constrained RL, what is it ?

**RL**

$$\pi^* = \arg\max_{\pi \in \Pi} J_R(\pi)$$

find the best policy

**Constrained RL**

$$\pi^* = \arg\max_{\pi \in \Pi} J_R(\pi), \text{ s.t. } J_{C_k}(\pi) \leq d_k \,, \; k = 1, \ldots, K$$

find the **best policy subject to some constraints**

# Constrained RL, what is it ?

**RL**

$$\pi^* = \arg\max_{\pi \in \Pi} J_R(\pi)$$

find the best policy

**Constrained RL**

$$\pi^* = \arg\max_{\pi \in \Pi} J_R(\pi), \text{ s.t. } J_{C_k}(\pi) \leq d_k \, , \; k = 1, \ldots, K$$

find the best policy subject to some constraints

Several CRL methods exist, we focus on Lagrangian ones

# Constrained RL, what is it ?

**RL**

$$\pi^* = \arg\max_{\pi \in \Pi} J_R(\pi)$$

find the best policy

**Constrained RL**

$$\pi^* = \arg\max_{\pi \in \Pi} J_R(\pi), \text{ s.t. } J_{C_k}(\pi) \leq d_k \,, \; k = 1, \ldots, K$$

find the best policy subject to some constraints

Several CRL methods exist, we focus on Lagrangian ones

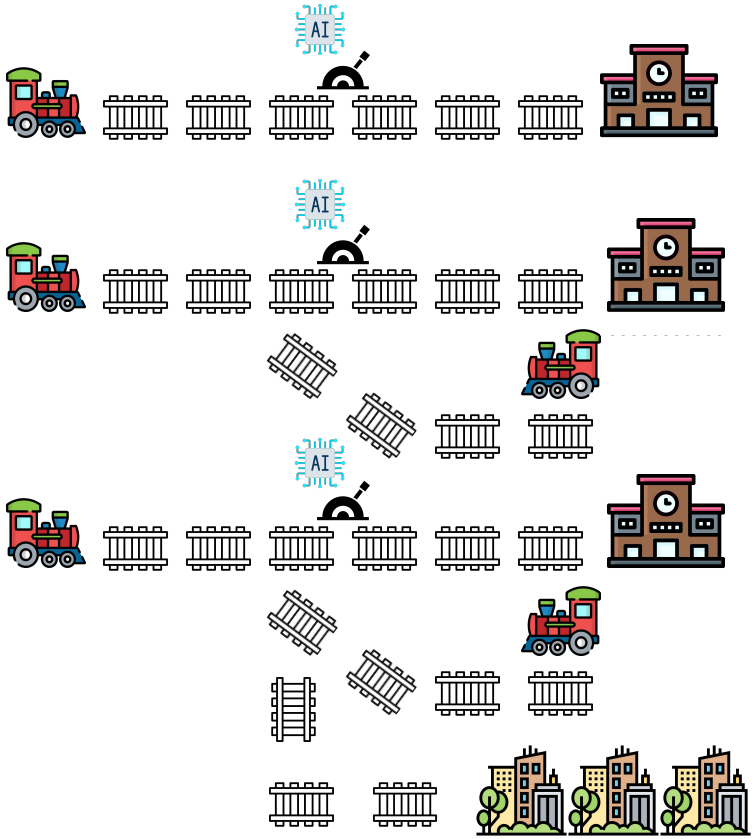
$$\max_{\pi} \min_{z_{1:K} \geq 0} \mathcal{L}(\pi, \lambda)$$

$$\mathcal{L}(\pi, \lambda) = \lambda_0 J_R(\pi) - \sum_{k=1}^{K} \lambda_k (J_{C_k}(\pi) - d_k)$$

**Lagrangian-CRL**

# Sequential Constraint Injection for Constrained Multi-Agent Reinforcement Learning

# Sequential Injection




Be **on time** !

Done !

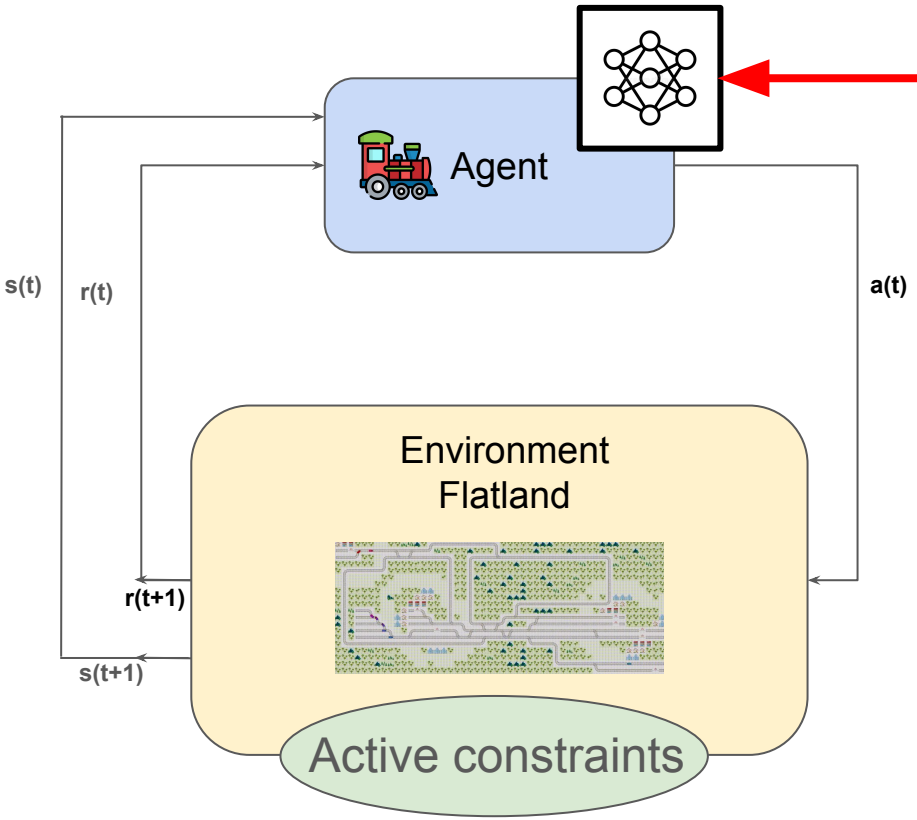
Be **on time**
and **no collision** !

Done !

Be **on time**, **no collision** and **avoid populated area** !

Done !

# Sequential Injection - Update Policy

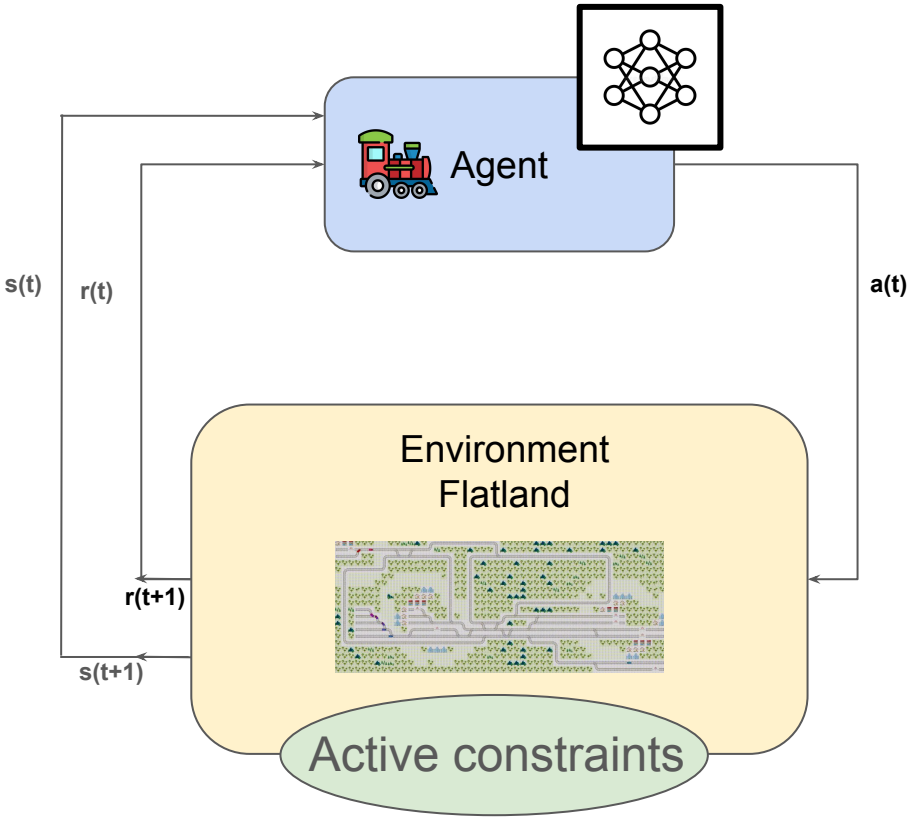



$$J_R(\pi) \quad J_{C_k}(\pi) \rightarrow \text{RL functions} \rightarrow \mathcal{L}_R^{PPO} \quad \mathcal{L}_{C_k}^{PPO}$$

$$\mathcal{L}_{total}^{PPO} = \mathcal{L}_R^{PPO} + \lambda_k \mathcal{L}_{C_k}^{PPO}$$

Inactive constraints

# Sequential Injection - Update Multipliers

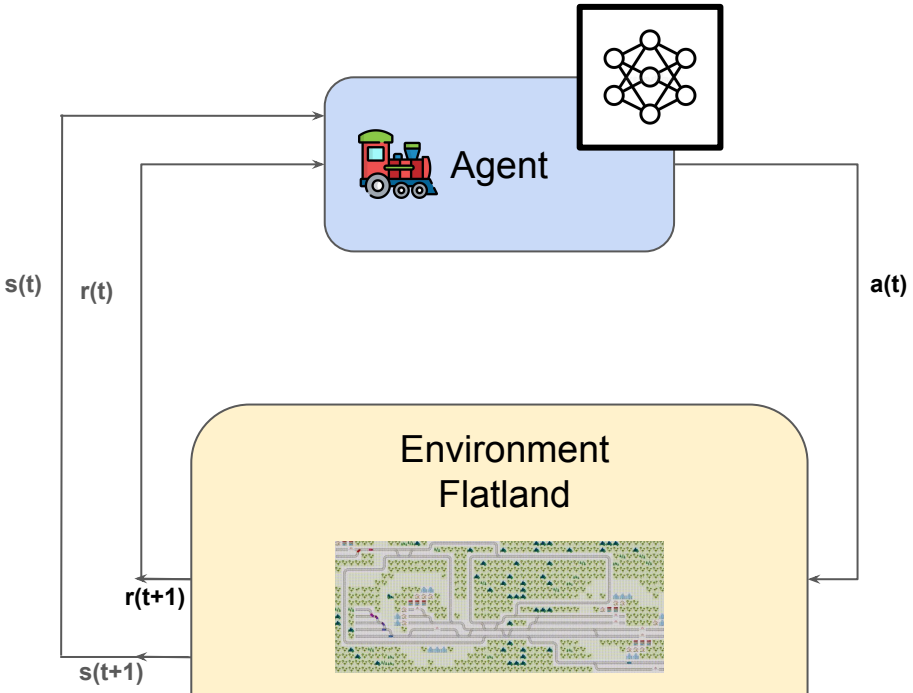



$$z_k^{(t+1)} \leftarrow z_k^{(t)} + \alpha(d_k - \hat{J}_k(\pi))$$

$$\lambda = softmax(z)$$

Inactive constraints

# Sequential Injection - Update active constraints set




Does our policy respected the active constraints during the last x training step ?

Yes

No

What inactive constraints is the most violated ?

keep unchanged

Add it to the active set

Active constraints

Inactive constraints

# Results

# Experimental setup

Policies:
AI SOTA: World Model + PPO + Imitation learning to max reward (Bourgeat et al.)
CRL: AI SOTA + All constraints at once
Seq-CRL: AI SOTA + sequential constraint injection

Metrics:
Reward: %of trains at destination at end of simulation
Deadlock(1%): %of trains deadlocked at end of simulation
Noise disturbance(5%): %train going through other's trains station during simulation
Time stopped(25%): %trains stopping voluntarily during simulation
Training time in hours

Means on 100 instances of 10 trains

# 1st Results

10 trains in Flatland Environment

| policy | reward | %deadlocks (<1%) | %noise disturbance (<5%) | %time stopped (<25%) | training time |
|---|---|---|---|---|---|
| AI SOTA | 84 | 2,5 | 2,2 | 30,1 | ~220h |
| CRL | 82 | 4,6 | 1,1 | 21,7 | ~30h |
| Seq-CRL | 91 | 1,3 | 0,6 | 21,3 | ~28h |

# 1st Results

10 trains in Flatland Environment

| policy | reward | %deadlocks (<1%) | %noise disturbance (<5%) | %time stopped (<25%) | training time |
|---|---|---|---|---|---|
| AI SOTA | 84 | 2,5 | 2,2 | 30,1 | ~220h |
| CRL | 82 | 4,6 | 1,1 | 21,7 | ~30h |
| Seq-CRL | 91 | 1,3 | 0,6 | 21,3 | ~28h |

**Best reward**

# 1st Results

10 trains in Flatland Environment

| policy | reward | %deadlocks (<1%) | %noise disturbance (<5%) | %time stopped (<25%) | training time |
|---|---|---|---|---|---|
| AI SOTA | 84 | 2,5 | 2,2 | 30,1 | ~220h |
| CRL | 82 | 4,6 | 1,1 | 21,7 | ~30h |
| Seq-CRL | 91 | 1,3 | 0,6 | 21,3 | ~28h |

**Constraints most respected but not perfectly**

# 1st Results

10 trains in Flatland Environment

| policy | reward | %deadlocks (<1%) | %noise disturbance (<5%) | %time stopped (<25%) | training time |
|---|---|---|---|---|---|
| AI SOTA | 84 | 2,5 | 2,2 | 30,1 | ~220h |
| CRL | 82 | 4,6 | 1,1 | 21,7 | ~30h |
| Seq-CRL | 91 | 1,3 | 0,6 | 21,3 | ~28h |

**Training time even lower**

# Next steps

**1- New criteria: Gradient Similarities (in progress)**

Every constraint has a gradient

Constraints can be:

- similar (their gradients push the policy in the same direction)
- conflicting (opposite direction)
- independent

➡ Add the inactive constraint the most different to the active ones

# Next steps

**1- New criteria: Gradient Similarities (in progress)**

Every constraint has a gradient

Constraints can be:

- similar (their gradients push the policy in the same direction)
- conflicting (opposite direction)
- independent

➡ Add the inactive constraint the most different to the active ones

**2- Validate on other environments (Omnisafe)**

# Next steps

**1- New criteria: Gradient Similarities (in progress)**

Every constraint has a gradient

Constraints can be:

- similar (their gradients push the policy in the same direction)
- conflicting (opposite direction)
- independent

➡ Add the inactive constraint the most different to the active ones

**2- Validate on other environments (Omnisafe)**

**3- Theoretical proof that learning is stabilized (any help is welcome)**

# Take-home messages

- Railway networks are **hard to solve**
- There are a lot of **physical** and **business constraints**
- **Constrained reinforcement learning** is an interesting approach
- Hard to stabilize
- **Sequential constraint injection** is promising



POLYTECHNIQUE MONTRÉAL
TECHNOLOGICAL UNIVERSITY

CORS · SCRO

SNCF